UNIVERSITY OF GHANA
Department of Computer Science

DCIT 208 — Software Engineering

Week 4 · Individual Assignment 1 (IA1)

Personal System Request-Flow Diagram

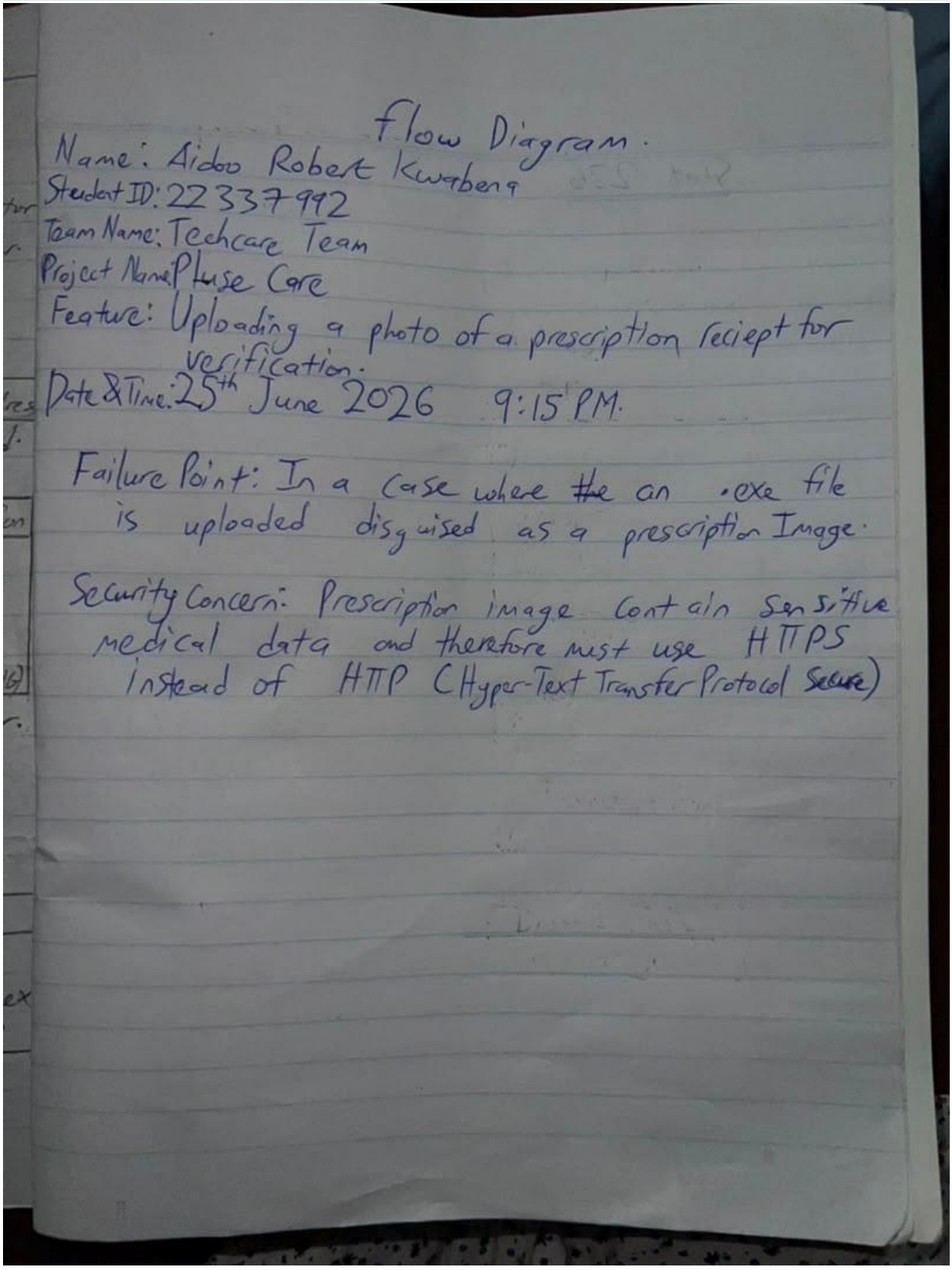
Course Code:	DCIT 208 — Software Engineering
Assignment:	IA1 — Personal System Request-Flow Diagram
Full Name:	Aidoo Robert Kwabena
Student ID:	22337992
Team Name:	Techcare Team
Project Name:	PluseCare
Chosen Feature:	Prescription Receipt Upload for Verification
Due Date:	Friday, 26 June 2026 — 5:00 PM Ghana Time
Date Submitted:	26th June 26, 2026
Team Repo URL:	https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-23techcare
Individual Repo:	https://github.com/AJ2025LEG/DCIT208-Client-Project-2026-Aidoo-Robert-Kwabena
YouTube Link:	https://youtu.be/x2vwXkWmbqo?is=v9ZiMQbqLmnf_oeG

AI Disclosure Statement

I used claude to help me better understand te task given and help in part of the layout of my final work.

Date	Tool	Prompt Summary	Output Used	What Was Rejected	Verification
25th June 26, 2026	Claude (Anthropic)	Help structure my assignment	Structure	Generic examples not related to our project	Read, understood, reworded and adapted to my own project context

SECTION A — Request-Flow Diagram



Flow Diagram.

Name: Aidoo Robert Kwabena
Student ID: 22337992
Team Name: Techcare Team
Project Name: Pluse Care
Feature: Uploading a photo of a prescription reciept for verification.
Date & Time: 25th June 2026 9:15 PM.

Failure Point: In a case where ~~the~~ an .exe file is uploaded disguised as a prescription Image.

Security Concern: Prescription image contain sensitive medical data and therefore must use HTPS instead of HTP (Hyper-Text Transfer Protocol ~~Secure~~)

Uploading a photo of a prescription reciept for verification.

Customer - Smartphone / Laptop

⇓ Types Uniform Resource Locator in browser search bar.

Browser - www.pharmacyname.com/upload-prescription

⇓ Uniform Resource Locator

DNS Server - Converts Uniform Resource locator to IP address

⇓ now the server is located.

HTTPS POST REQUEST - /upload-prescription

Details of the Data Sent:
- Customer Name for records
- Phone number for keeping records
- Prescription Image File (format JPG/PNG)

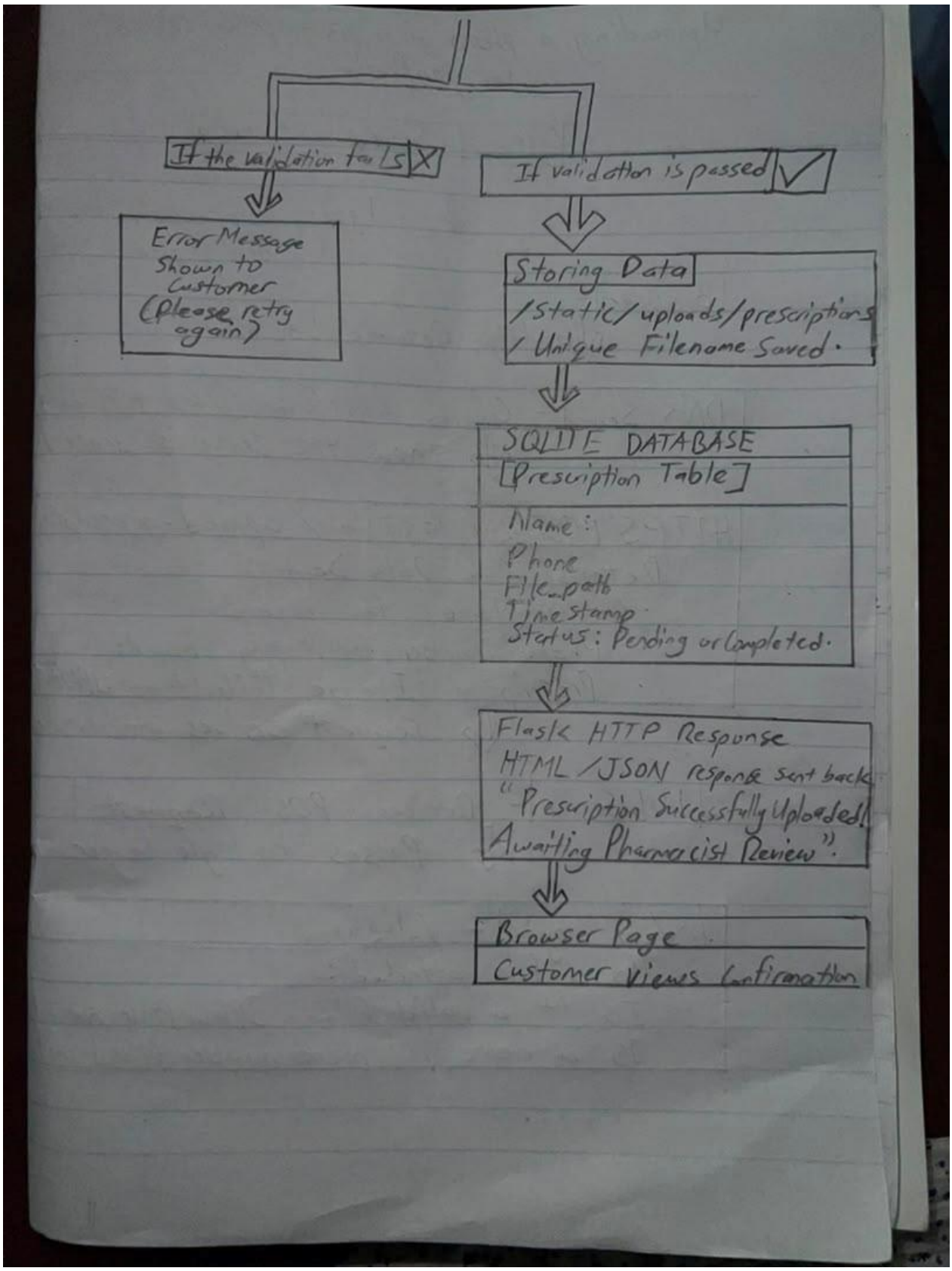
⇓ request arrives at server.

Web Server - Recieves POST Request

⇓ Passes to logic layer

Web Logic - Validation

Is a file attached?
Is it a valid file. a JPG or PNG not .exe?
Is a name and phone number filled in?


If the validation fails X
If validation is passed ✓
Error Message Shown to Customer (Please retry again)
Storing Data
/static/uploads/prescriptions
/ Unique Filename Saved.
SQLITE DATABASE
[Prescription Table]
Name:
Phone
File_path
Timestamp
Status: Pending or Completed.
Flask HTTP Response
HTML / JSON response sent back:
"Prescription Successfully Uploaded! Awaiting Pharmacist Review".
Browser Page
Customer views Confirmation

SECTION B

1. Introduction

Our team project is a pharmacy web application developed for a newly established pharmacy. The goal of the application is to provide customers with a convenient way to order medications online and request consultations with pharmacists via phone call, without having to physically visit the pharmacy. For this individual assignment, I have chosen the **Prescription Receipt Upload for Verification** feature as the basis of my request-flow diagram. This feature is important because certain medications cannot legally be dispensed without a valid prescription from a licensed medical doctor. By allowing customers to upload a photo of their prescription receipt online, the pharmacy can verify it before processing a drug order, ensuring legal compliance and patient safety.

2. The Request Flow — Step by Step

Step 1 — The User and Their Device

The customer accesses the pharmacy website using a browser such as Chrome or Firefox on a smartphone or laptop. Since our team is building a web application and not a native mobile app, all interactions happen through the browser. A smartphone is especially common for this feature since customers can take a photo of their prescription directly from their phone camera and upload it immediately.

Step 2 — Typing the URL and DNS Resolution

The customer navigates to the prescription upload page, for example www.pharmacyname.com/upload-prescription. The browser does not immediately know where this website is hosted on the internet. It first contacts a DNS (Domain Name System) server, which functions like an internet phonebook — converting the human-readable URL into a numeric IP address pointing to our Flask web server.
The browser then uses that IP address to establish a connection to the server.

Step 3 — Sending the HTTPS Request

The customer selects their prescription photo from their device using the file upload input on the form, fills in their name and phone number, and clicks the 'Upload Prescription' button. The browser then sends an HTTPS POST request to the server. This request is a multipart/form-data request — a special format used when a form contains both regular text fields and a file. We use HTTPS rather than plain HTTP because it encrypts all data in transit, including the image file itself. This is critical in a pharmacy context because a prescription receipt may contain sensitive personal and medical information.

Step 4 — The Flask Server Receives the Request

Our team plans to use Python Flask as the backend web framework. Flask runs continuously, listening for incoming requests. When the POST request arrives at the /upload-prescription route, Flask receives both the text fields and the uploaded image file, then passes them through the application logic.

Step 5 — Validation (Application Logic)

Before saving anything, Flask validates the submission. It checks that the customer has actually attached a file, that the file is an accepted image format such as JPG or PNG and not a potentially harmful file type like an executable, that the file size does not exceed a reasonable limit, and that the customer's name and phone number fields are not empty. This step is essential for both security and usability — the pharmacy should only receive valid, readable prescription images.

Step 6 — File Storage and Database Interaction

If the file passes all validation checks, Flask saves the uploaded image to a designated folder on the server, for example /static/uploads/prescriptions/. A unique filename is generated — such as using the customer's ID and a timestamp — to avoid overwriting existing files. Flask then saves a record to the SQLite database, inserting a

new row into the prescriptions table. This record stores the customer's name, phone number, the file path of the saved image, the upload timestamp, and a verification status field set to 'Pending' by default.

Step 7 — The Response

After the file and record are successfully saved, Flask sends an HTTP response back to the browser — an HTML page or JSON message such as 'Your prescription has been uploaded successfully. A pharmacist will review it and contact you shortly.' The customer sees this confirmation in their browser, completing the full request-response cycle. The admin can then log in separately to view and verify the uploaded prescription before approving the drug order.

3. Answering the 12 System Design Questions

#	Question	Answer
1	**What device is the user on?**	The customer is most likely on a smartphone — since they can photograph their prescription directly — but the feature also works on a laptop where the customer uploads an already-saved image file.
2	**Is it a browser or mobile app?**	It is a browser-based web application. Our team is building a website, not a native Android or iOS app.
3	**What URL, route, or endpoint is involved?**	The endpoint is POST /upload-prescription on the Flask server.
4	**What protocol is used?**	HTTPS is used to encrypt all data in transit, including the prescription image file, which may contain sensitive personal and medical information.
5	**What data is sent?**	The form submits a multipart/form-data request containing: the customer's full name, phone number, and the prescription image file (JPG or PNG).
6	**What validates the request?**	Flask performs server-side validation — checking that a file was attached, that it is an accepted image format (JPG or PNG), that the file size is within an acceptable limit, and that the name and phone number fields are not empty.
7	**What database table or service is touched?**	The prescriptions table in the SQLite database receives a new inserted record: customer details, the file path, the upload timestamp, and a default verification status of 'Pending'.
8	**What response comes back?**	Flask returns a success HTML page or JSON response confirming the prescription has been uploaded and is awaiting pharmacist review.
9	**What happens if validation fails?**	Flask returns an error response and the browser displays a message such as 'Please attach a valid image file (JPG or PNG)' or 'File size is too large.' Nothing is saved and the customer is prompted to try again.
10	**What happens if the database is down?**	Flask catches the error using Python exception handling (try/except) and returns: 'We are currently unable to process your upload. Please try again later.' The application does not crash.

11	**What if 500 users upload at once?**	SQLite handles one write at a time and may slow or lock under heavy load. Additionally, storing many large image files simultaneously could fill server disk space. For a small pharmacy this is manageable, but scaling would require PostgreSQL and a dedicated file storage service.
12	**What security or privacy concern exists?**	Prescription receipts reveal health conditions and doctor details. All uploads must go over HTTPS only; file types must be strictly validated to prevent malicious uploads; saved files must be stored in a protected folder not directly publicly accessible; and patient data must never be exposed through any public tool or repository.

4. Reflection

The team currently plans to use Python Flask and SQLite for the backend of the pharmacy web application. If the team later decides to switch to a different framework, database, or file storage solution, this request-flow document will be updated accordingly. One thing I still need to learn and understand better is how Flask handles file uploads securely and what additional libraries or techniques exist to validate image files more thoroughly beyond just checking the file extension.

End of Submission